

Scaling out NUMA-Aware Applications with RDMA-Based Distributed Shared Memory

Yang Hong, Yang Zheng, Fan Yang, Bin-Yu Zang, *Distinguished Member, CCF, Member ACM, IEEE*
Hai-Bing Guan, *Senior Member, CCF, ACM, IEEE*, and
Hai-Bo Chen*, *Distinguished Member, CCF, Senior Member, ACM, IEEE*

Shanghai Key Laboratory for Scalable Computing Systems, Shanghai Jiao Tong University, Shanghai 200240, China

E-mail: {yang.hong, tomsun.0.7, Fan_Yang, byzang, hbguan, haibochen}@sjtu.edu.cn

Received June 13, 2018; revised November 21, 2018.

Abstract The multicore evolution has stimulated renewed interests in scaling up applications on shared-memory multi-processors, significantly improving the scalability of many applications. But the scalability is limited within a single node; therefore programmers still have to redesign applications to scale out over multiple nodes. This paper revisits the design and implementation of distributed shared memory (DSM) as a way to scale out applications optimized for non-uniform memory access (NUMA) architecture over a well-connected cluster. This paper presents MAGI, an efficient DSM system that provides a transparent shared address space with scalable performance on a cluster with fast network interfaces. MAGI is unique in that it presents a NUMA abstraction to fully harness the multicore resources in each node through hierarchical synchronization and memory management. MAGI also exploits the memory access patterns of big-data applications and leverages a set of optimizations for remote direct memory access (RDMA) to reduce the number of page faults and the cost of the coherence protocol. MAGI has been implemented as a user-space library with pthread-compatible interfaces and can run existing multithreaded applications with minimized modifications. We deployed MAGI over an 8-node RDMA-enabled cluster. Experimental evaluation shows that MAGI achieves up to 9.25x speedup compared with an unoptimized implementation, leading to a scalable performance for large-scale data-intensive applications.

Keywords distributed shared memory (DSM), scalability, multicore evolution, non-uniform memory access (NUMA), remote direct memory access (RDMA)

1 Introduction

The multicore evolution has drastically increased the number of CPU cores per chip. Together with the evolving of processor interconnect technologies, commodity multicore servers are increasingly built with cache-coherent non-uniform memory access (cc-NUMA) architectures to accommodate more CPU cores. AMD has recently announced its Naples^① with “die-NUMA” technology that packs four NUMA domains within a single socket. The increase in core

count and complexity of core topology has also driven commodity software to be multicore-optimized and NUMA-aware through exploiting the heterogeneous cache/memory hierarchy and reducing unnecessary data sharing^[1–7].

While there have been a lot of techniques to optimize multithreaded applications for cc-NUMA, programmers still have to redesign their systems with distinct optimization strategies when programming a cluster of machines. This study tries to answer the question of how to inherit and reuse existing optimizations on

Regular Paper

This work was supported in part by the National Key Research and Development Program of China under Grant No. 2016YFB1000500, the National Natural Science Foundation of China under Grant No. 61572314, and the National Youth Top-Notch Talent Support Program of China.

*Corresponding Author

① Advanced Micro Devices, Inc. NUMA topology for AMD EPYC™ naples family processors. <https://developer.amd.com/wp-content/resources/56308-NUMA%20Topology%20for%20AMD%20EPYC%E2%84%A2%20Naples%20Family%20Processors.PDF>, Oct. 2018.

©2019 Springer Science + Business Media, LLC & Science Press, China

multicore and NUMA systems to scale out applications across a cluster without massive re-engineering.

Software distributed shared memory (DSM)^[8] is a cost-effective solution to support shared memory abstraction for parallel programming on a cluster of commodity servers. Due to the intriguing promise, implementing scalable and efficient DSM systems has been a pursuit of researchers for decades^[8–16]. However, prior software DSM systems suffer from high access latency and poor scalability^[8–10], or/and only present a partial global shared memory^[17–20]. These limitations inhibit the use of DSM systems in scaling existing multi-threaded applications with trivial effort.

We revisit software DSM systems in the context of emerging hardware and software. Our study of the evolution of hardware uncovers that the performance of processors and network interfaces has undergone radical changes. The latency of network interfaces has been reduced from hundreds of microseconds^[15] to around 2 μ s^[21] while the latency of translation look-aside buffer (TLB) shutdown on one processor changes from tens of microseconds^[16] to less than 1 μ s^[22]. Meanwhile, applications are increasingly adopting architecture-aware designs to meet the growing complexity of program logic and the demand for scalability. Typically, NUMA-aware optimizations help applications to fully exploit the asymmetric multicore architecture. As a result, the changes in both hardware and software refresh the outdated understanding of DSM systems and motivate researchers to rethink the designs.

This study presents MAGI which is a software DSM system employing a hybrid consistency model to practically scale out single-node data-intensive applications across a cluster of commodity multicore servers. MAGI exploits the data locality in coarse-grained data-parallel jobs and the low-latency network interfaces to achieve good scalability. MAGI is designed to port applications to run on the DSM system with trivial modifications. To this end, MAGI provides an abstraction of a NUMA-style multicore system. MAGI implements compatible memory and thread APIs along with a set of efficient synchronization primitives to minimize porting efforts.

MAGI’s hybrid consistency model combines sequential consistency (SC) with application-controlled consistency relaxation. Sequential consistency provides programming compatibility with single-node systems, while consistency relaxation allows applications to directly access remote data, temporarily overriding the protocol constraints. This design is based on the analysis of memory access patterns in modern big-data

applications and coincides with the intuitions of past DSM optimizations to relax the memory consistency constraints. The hybrid scheme offers a low-overhead sequential consistency implementation as well as the flexibility to pursue higher efficiency under programmers’ control.

MAGI implements sequential consistency with a two-level approach where inter-node memory accesses are coordinated by the DSM protocol, while intra-node data sharing leverages the hardware shared memory mechanism. Because intra-node communication is orders of magnitude faster than inter-node communication, this approach naturally results in the NUMA-like asymmetry. Therefore, MAGI is a good fit for NUMA-aware applications which feature intra-node data locality and optimized inter-node synchronizations.

To reduce the page fault overhead in inter-node data sharing, MAGI introduces a set of optimizations to the protocol implementation. The first technique is speculative fault. Assuming sequential memory access patterns in applications, MAGI tries to request more pages when handling a fault. The second technique is batched TLB shutdowns which is designed to ease the scalability issue of translation coherence on symmetric multiprocessing (SMP) machines. The third technique is polling-based request handling. It dedicates cores to handling protocol requests for low latency. The three optimizations together enhance the performance of sequential consistency with efficient data sharing across SMP nodes.

To facilitate application-controlled consistency relaxation, MAGI provides protocol-bypassing operations to allow direct access to remote data. Specifically, applications instruct MAGI to fetch data in specific DSM regions from remote nodes within custom synchronization mechanisms (e.g., locks, barriers, and semaphores). The low-latency one-sided remote direct memory access (RDMA) primitives further enhance the efficiency of data transfers.

While the design of MAGI can be applied to operating systems and virtual machine systems^[10] as well, we currently implement MAGI as a user-level library for simplicity. Specifically, MAGI implements fault handling in the user-space signal handler and leverages the user-space RDMA APIs for efficiency.

To study the performance of MAGI, we have conducted a set of evaluations on an 8-node RDMA-enabled cluster with a total of 160 Intel® Xeon E5-2650 v4 cores. We analyze the effect of protocol optimizations with detailed breakdowns of the exe-

cution overhead. We evaluate the scalability of MAGI using small-scale parallel programs as well as large-scale data-intensive workloads. Performance evaluations demonstrate that MAGI can scale out parallel applications across 160 cores with sequential consistency. With certain modifications to the applications, protocol-bypassing operations can further reduce the overhead of page faults and enhance the throughput.

In summary, this study makes the following contributions.

- A DSM design that employs a hybrid consistency model to provide low-overhead sequential consistency and support application-controlled consistency relaxation.
- A set of optimizations to practically reduce the overhead of the DSM protocol by exploiting RDMA-enabled network interfaces and memory access patterns of emerging applications.
- An implementation of the DSM system with the proposed optimizations and a set of evaluations that shows NUMA-aware applications can achieve up to 9.25x speedup with the proposed optimizations.

The next section revisits the trends in both software and hardware and briefly reviews the classic Ivy DSM protocol. Section 3 describes the basic design of MAGI, followed by a set of optimizations in Section 4. Section 5 takes a detailed look at the implementation details, and Section 6 presents a quantitative evaluation of the MAGI system using a set of benchmarks with different scales. Section 7 discusses the current limitations as well as trends in RDMA supports and its impact on MAGI. Finally, Section 8 describes related work with Section 9 to conclude.

2 Background and Motivations

2.1 Hardware and Software Trends

Scale-up Designs Are Power-Limited. The end of Dennard scaling law^[23] means that we can no longer retain the power density with smaller transistors, which limits the increase of compute density within a single chip^[24]. This might partially explain the reason why the number of cores within a chip has increased much slower in recent years^[25]. To overcome single-chip scalability, commodity multicore architectures have adopted cc-NUMA designs where multicore

processors are connected by high-speed interconnect (e.g., QuickPath Interconnect (QPI) and HyperTransport) within a single machine. Each processor, also known as a NUMA node, directly accesses the local memory via their own memory controller with low latency and accesses remote memory from other NUMA nodes through the off-chip interconnect with high latency.

Nevertheless, the topology complexity of processor interconnect still leads to an incommensurate cost scaling with the number of sockets for a single machine. For example, an 8-socket server machine is usually 10x (instead of 4x) the price of a 2-socket server machine with the proportional hardware configuration. Furthermore, the power limit and the incommensurate cost scaling similarly apply to dynamic random-access memory (DRAM) such that it is less cost-effective to scale up a single machine in an unconstrained way. Thus, it is reasonable to see the common server configurations with 2–4 sockets in data centers.

Emerging Applications Become Popular. Data-intensive applications (e.g., MapReduce^[26,27], graph analytics^[6,28] and deep learning systems) have become attractive to the research community and the industry. Different from classic parallel workloads, those applications have less frequent data sharing and synchronizations to achieve better scalability on multicore and NUMA systems. The coarse-grained data-parallel pattern helps mitigate the increased communication cost and performance gap among computer system components. It also motivates a revisit of scaling out applications with DSM systems. The improved locality and the reduced communication expose opportunities to bypass the shortcomings of DSM which would otherwise limit the scalability.

RDMA-Enabled Network Is Increasingly Deployed. RDMA has been popular in the high-performance computing (HPC) field for many years, granting applications low-latency and high-throughput accesses to memory on remote hosts. Recently, RDMA-enabled network interfaces^{②③} are easily accessible and deployed in data centers at scale^[29], providing fast and inexpensive inter-machine communication for data center applications.

Every RDMA request is described as a work request. Each work request, if configured as signaled, finishes

② InfiniBandSM Trade Association. InfiniBandTM architecture specification. <https://cw.infinibandta.org/document/dl/7161>, Oct. 2018.

③ InfiniBandSM Trade Association. Supplement to InfiniBandTM architecture specification. <https://cw.infinibandta.org/document/dl/7781>, Oct. 2018.

with generating a work completion (WC) in the completion queue (CQ). Applications must periodically poll the CQs to drain the outstanding CQ elements. The RDMA stack provides two types of primitives, namely one-sided and two-sided operations. With one-sided operations, applications directly access the memory of remote hosts without involving the kernel and remote CPUs. Two-sided operations have similar programming interfaces to socket APIs. The server posts an RECV work request before the client posts an SEND work request to the server. While the kernel is still out of the critical path, the receiver has to spare CPU shares to explicitly poll the CQ for incoming messages.

With either type of operations, the measured round-trip latency of an RDMA message is about 2 μ s, approaching the latency of some operations inside CPUs. For example, an inter-processor interrupt to 20 cores takes about 1 μ s according to our measurement. This observation renders it meaningful to re-evaluate the performance of DSM systems renewed hardware features.

2.2 Memory Consistency Models

The first DSM system^[8] implements sequential consistency. The basic idea is implementing a software coherence protocol inspired by the hardware shared memory mechanism. Plenty of subsequent systems show that the strong ordering constraints on memory accesses enforced by sequential consistency result in poor scalability. Subsequent DSM systems (e.g., [9, 11, 13–16, 30]) widely adopt various relaxed consistency models to reduce the amount of communication and use multiple-write protocols to handle the false sharing problem.

However, as the x86 architecture becomes popular and widely deployed along with the software ecosystem, not only does the architecture lack support for explicitly expressing shared memory accesses, but also applications usually adopt custom synchronization mechanisms. Therefore, to scale out single-node parallel applications to run on a cluster, DSM systems based on release consistency require non-trivial modifications to original applications. In contrast, sequential consistency enforces strong memory ordering constraints and is compatible with the total store ordering (TSO) memory model used by commodity x86 machines. This study tries to scale out existing NUMA-aware data-intensive applications with minimum efforts using DSM across commodity clusters. Therefore, MAGI chooses sequential consistency as the basic consistency model.

2.3 Ivy Protocol

Ivy^[8] is a DSM system that provides sequential consistency memory model. It implements a single-writer-multiple-reader protocol that resembles the CPU cache coherence protocol. Ivy resolves memory access conflicts at page granularity. Each page can be in any of the three states: read-only (RO), read-write (RW), and invalid (IV).

Fig.1 shows how Ivy maintains memory consistency. Processor P_0 triggers a page fault upon reading a non-present page and enters the read fault handling routine. It sends a read query (RQ) to the manager to request the latest copy of the page. In response to the read query, the manager sends a read forward message to the page owner P_1 and temporarily blocks concurrent requests for the same page until a read confirm message is received from P_0 . The manager maintains the state of all DSM pages with a directory. The owner P_1 responds to read forward by removing the write permission of its own copy and sending the page data to P_0 in a read data message. Finally P_0 confirms data reception to the manager by sending a read confirm message. Write faults are handled similarly as in Fig.1(b).

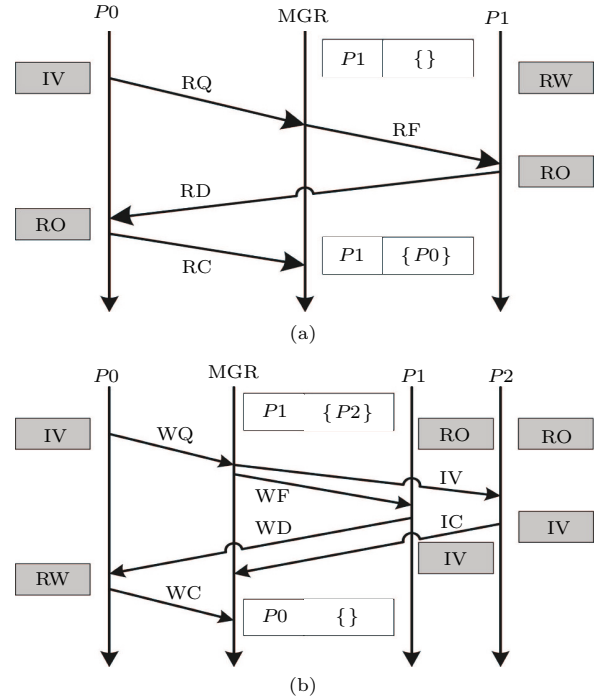


Fig.1. Fault handling in Ivy. MGR represents the directory manager. Entities communicate with different messages: read query (RQ), read forward (RF), read data (RD), read confirm (RC), write query (WQ), write forward (WF), invalidate (IV), invalidate confirm (IC), write data (WD), and write confirm (WC). (a) Read fault. (b) Write fault.

The improved version of Ivy distributes the load on the centralized manager to multiple distributed instances so that the manager does not become the bottleneck. Our system implements the improved Ivy protocol with optimizations by exploiting advances in hardware and emerging patterns in applications.

3 MAGI Prototype System

MAGI is a prototype DSM system for a cluster of commodity servers with RDMA network interfaces. We implement the system as a user-space library along with a kernel module for manipulating page tables. Fig.2 shows the framework of MAGI.

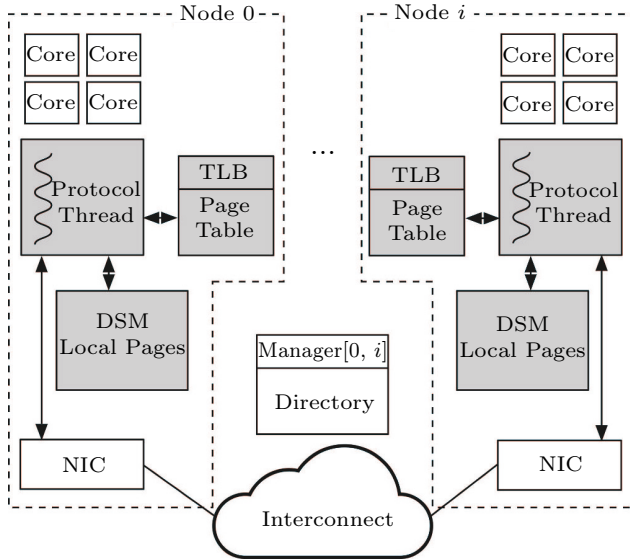


Fig.2. Framework of MAGI.

MAGI implements a hybrid memory consistency model which provides sequential consistency by default but also allows programmers to explicitly override the consistency protocol for efficiency. Following the design of Ivy, the sequential consistency protocol synchronizes pages among nodes with the invalidation approach such that only one writer is allowed for each page.

NUMA Abstraction. MAGI provides applications with a cc-NUMA abstraction. Application threads can obtain the runtime CPU topology and create other threads with desired node affinity. Currently, MAGI allows a thread to be bound to a desired node at creation time, but does not support thread migration across nodes. Providing dynamic node affinity is left for future work.

Under the hood, MAGI adopts the clustering design from previous systems (e.g., [15, 16, 31]). Intra-node cores leverage the local hardware cache coherence mechanism to efficiently share the memory data, while inter-node data sharing follows the DSM protocol. In practice, application threads on the same node share the same page table.

Memory Fault Detection. MAGI uses the virtual memory mechanism to detect conflicting memory accesses. Specifically, modern processors translate virtual addresses to physical addresses via the combination of page tables and TLBs for permission checking and address space virtualization. MAGI leverages the permissions in page table entries to denote node-local states of DSM pages, namely, invalid, read-only, and read-write. A page access that fails permission check triggers a page fault exception.

Although large pages may help reduce the number of page faults in data intensive applications, we use 4 KB page size instead of 2 MB because using small pages is more flexible. While our scheme indeed degrades the performance of some applications which are optimized for exploiting huge pages, our approach can avoid the overhead of increased false sharing and transparent huge pages manipulations^④ (THP)^[32].

Changing the permission of one page on a single core is implemented with page table entry modifications followed by a local TLB shutdown (e.g., executing `invlpg` instruction or switching the `CR3` register). However, in an SMP system, all processors must have a consistent view of the address space mappings. Current x86 architecture does not provide hardware support for translation coherence. Therefore, operating systems rely on inter-processor interrupts (IPIs) to invalidate TLB entries on other processors after modifying page table entries. This mechanism is also known as TLB shutdown.

Distributed Directories. MAGI adopts the directory-based approach to maintain the global states of DSM pages and resolve conflicting accesses. The directory is distributed across all nodes to reduce the contention on a centralized manager. Each node spawns a directory manager process for handling DSM page requests for a subset of the address space. These managers are designed to be responsive such that they send out messages in response to incoming requests in a match-action pattern. During our experiments, the CPUs dedicated for managers are never saturated. Therefore, the

^④ Arcangeli A. Transparent huge page support. <https://www.linux-kvm.org/images/9/9e/2010-forum-thp.pdf>, Oct. 2018.

directory managers do not become the bottleneck of the protocol.

4 Optimizations

Conventional DSM systems are usually believed to have poor performance due to the high cost of maintaining the memory consistency. MAGI proposes a set of optimization techniques to reduce the overhead of the protocol in three aspects which are marked in grey in Fig.2.

4.1 Speculative Faults

Data-intensive applications are usually optimized to exploit the spatial and temporal locality of data to minimize the CPU delays caused by the latency gap between memory hierarchy components. When a thread in such an application triggers a page fault in DSM, chances are that it will also access pages nearby in the near future. Those applications typically feature sequential data access patterns with large memory footprints to make full use of caches and prefetching mechanisms provided by CPUs.

Therefore, inspired by prefetching mechanisms in CPUs, we propose speculative faults. When handling a page fault, the faulting processor should normally retrieve the up-to-date target page from the owner following the DSM protocol. With speculative faults, this processor will also attempt to proactively trigger additional page faults on N subsequent speculative pages (no more than the tunable maximum length M) other than the target page in the hope that the application will soon access the N pages. The faulting thread can resume execution as soon as the target page is ready while the data of speculative pages may asynchronously arrive. In practice, this approach overlaps the protocol operations on speculative pages with application logic. When threads on the same node access the speculative pages in the future, it is likely that these pages have been available due to exercising the protocol speculatively and no further page faults are needed. Even if a thread accesses the speculative pages before the asynchronous process completes, these pages are already on the fly and the latency of fault handling can be reduced.

Speculative page requests are made with the same access mode as the triggering fault (e.g., a write fault results in speculative write faults). The maximum length of speculative requests is tunable, and the actual number of prefetched pages might be smaller because the

speculation stops if the owner of the $(N+1)$ -th page differs from the previous N pages. The design decision is based on the observation that the interleaved ownership of consecutive pages implies complicated sharing patterns of the specific DSM region. Speculatively acquiring contiguous pages from different owners is prone to performance degradation due to additional false sharing. Meanwhile, the “next- N ” design simplifies the implementation with fewer messages.

While speculative execution is widely adopted in micro-architecture designs, it may not benefit DSM systems due to the risk of exacerbating false sharing. However, two key factors make it effective in MAGI. On the one hand, RDMA network makes for inexpensive speculation. Using low-latency RDMA messages, speculations can be made without affecting the general protocol latency. The high bandwidth also allows more data to be transferred. On the other hand, the data parallelism of emerging applications is a perfect fit for locality-based speculative page faults. Thus, the effect of this technique indeed depends on the memory access patterns, memory sharing granularity, the ratio of computation to memory access and memory sharing granularity.

Note that our speculation mechanism is different from the page prefetching mechanism in operating systems where the prefetching happens when data are loaded into memory for the first time. Our approach is designed for actively retrieving shared pages from other nodes, which constantly happens during the execution of data-intensive applications.

4.2 Batched TLB Shutdown

The two-level design of MAGI enables threads on the same node to share memory with the hardware cache coherence mechanism. To ensure that all the processors on one node have the same access permission on a page, we leverage the hardware paging support in current operating systems. While this approach makes intra-node communication significantly faster than inter-node communication, it introduces the problem of IPIs due to TLB shutdowns after modifying page tables.

We conduct an experiment on a single machine with two NUMA nodes by sending IPIs from one core to a varying number of target cores to execute an empty function. We start with two target cores because the Linux kernel handles differently for single-target IPIs. Fig.3 shows the latency of IPIs with the increasing num-

ber of target cores. If the target cores are idle, the latency of IPIs increases linearly with the number of targets. If remote cores are busy accessing memory, the cost of IPIs is even higher due to task scheduling and interconnect contention. The x2APIC architecture^⑤ can indeed reduce the overhead of IPIs with the clustering design, but the scalability problem remains. Therefore, building a scalable DSM system on large SMP machines is impractical, unless the overhead of TLB shootdowns can be reduced.

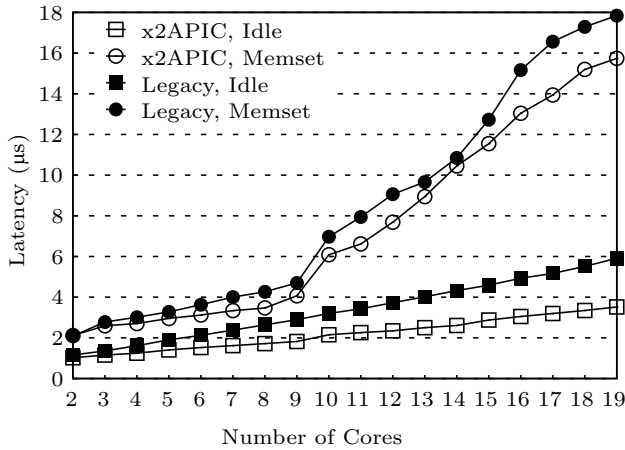


Fig.3. Scalability of TLB shootdowns.

We propose batched TLB shootdown to ease the overhead of TLB shootdowns caused by page permission elevation. When a page arrives as the result of a page request, MAGI does not immediately make it locally visible. Specifically, the corresponding page table entry is correctly updated without invalidating the TLBs of all other local processors. Expecting more pages to arrive within a short period, MAGI tries to batch the TLB shootdowns for multiple pages with a single IPI. This approach can greatly reduce the number of IPIs and the related overheads (e.g., context switches and interrupt handling). Note that only the TLB shootdowns for page permission elevation are batched while page invalidations are still carried out synchronously to guarantee sequential consistency.

The batching technique alone might increase the latency of a single page fault. But combined with the speculative faults mechanism, the temporal locality of consecutive page faults makes the overhead of batching negligible. Note that we try to batch TLB shootdowns of contiguous pages instead of sets of random

pages, and the reason is that faults triggered on scattered pages are typically from different execution contexts. Batching TLB shootdowns of such pages will increase the latency of individual faults without significant benefits. Although current operating systems typically adopt lazy TLB shootdowns^[22] to hide the latency of performing `munmap` system call, our approach cannot rely on this mechanisms. On the one hand, we cannot batch TLB shootdowns for permission downgrade due to violating sequential consistency constraints. On the other hand, we can only selectively batch TLB shootdowns for permission elevation to avoid false sharing.

4.3 Polling-Based Request Handling

RDMA provides two paradigms for users to receive incoming messages, namely, the event mechanism and the polling mechanism. The event mechanism provides a blocking system call to allow programs to wait on an event channel. When a message comes, the kernel gets notified via an interrupt from the network interface card (NIC) and wakes up the user-space process with a signal. This paradigm inevitably incurs high overhead of scheduling and context switches.

Therefore, MAGI chooses the polling mechanism and dedicates cores on each node (using the `sched_setaffinity` system call) for handling protocol requests. Only specific threads are scheduled to run on the dedicated cores to avoid CPU sharing. There are mainly two types of threads that require dedicated cores. We dedicate one core for the distributed manager thread on each node to minimize the processing time at managers so that they will not become the bottleneck. And we dedicate another core for the per-node thread which is created along with application threads for handling incoming protocol messages from managers and other nodes. This approach can notably speed up protocol handling at the cost of reduced available CPU cores for application threads.

4.4 Protocol-Bypassing Operations

While the aforementioned techniques try to optimize the implementation of sequential consistency, this consistency model is substantially inefficient in exploiting certain memory access patterns. Many data-intensive applications exhibit the unidirectional data sharing pattern where application threads share memory buffers in a producer-consumer model. These ap-

^⑤Intel Corporation. Intel® 64 architecture x2APIC specification. <https://software.intel.com/sites/default/files/managed/c5/15-/64-architecture-x2apic-specification.pdf>, Oct. 2018.

plications typically adopt the epoch-based design (e.g., phases in MapReduce and iterations in graph analytic frameworks). Each epoch fetches the data produced by the previous epochs and in turn puts the transformed results into a buffer for the next epochs. In this pattern, the pages are only modified by producers and the consumers only perform read-only accesses. However, the sequential consistency model based on page-level translation mechanism fails to efficiently capture this pattern. As a result, the unidirectional data pattern leads to ownership thrashing between nodes and redundant data transmission.

We propose protocol-bypassing operations to allow programmers to directly access DSM memory, overriding the constraints of sequential consistency. In practice, we provide the interface `direct-read` to allow data to be fetched from specific owner nodes and applied to the local DSM regions. The owner's copy is not invalidated; therefore no page faults will be triggered on the owner side. This operation is allowed with the premise that the local copies updated with direct-reads shall not be modified. The reason is that protocol-bypassing operations fetch pages without following the protocol such that the nodes holding the buffers from direct-read are not in the copy sets at managers. The consequence is that these nodes may not receive subsequent invalidations of such pages and future modifications to such pages will not be propagated. Once a thread finishes data transfers with proper synchronizations, it can switch back to sequential consistency by removing the permissions of those local copies.

The protocol-bypassing mechanism is proposed based on the observation that applications typically coordinate data transfers across phases or iterations with explicit barriers, making sequential consistency constraints unnecessary. By collecting fragmentary page faults which frequently interrupt computational tasks to a preplanned data access phase, MAGI can eliminate the page fault overhead (e.g., context switches and cache pollution), although the time of remote data access is inevitable in distributed applications. This observation also coincides with the intuition of release consistency. However, this technique is indeed a trade-off between the ease of programmability and efficiency, because the correctness of application logic depends on programmers to follow the premise of protocol-bypassing. Currently, MAGI only supports fetching remote data for simplicity.

5 Implementation

Modifications to Applications. MAGI requires the original single-node applications to be modified in two aspects. First, MAGI transparently integrates DSM protocol routines into the control flow of target applications. Specifically, MAGI performs fault handling in the user-space by registering as the callback function of the `SIGSEGV` signal to intercept page faults. Also, applications are statically linked with MAGI's library `libmthread` in place of `libpthread` to avoid dynamic linking issues. We use linker scripts to separate application contents from MAGI's library data by re-arranging the virtual addresses of sections of the binary files. MAGI transparently initializes DSM memory regions and RDMA connections before the `main()` function by putting setup code in the `.init_array` section of the binary files. Second, MAGI replaces C library routines with the distributed implementations in `libmthread`. As `libmthread` is statically linked to replace `glibc` and `libpthread`, our library provides implementations for frequently-used functions, such as `malloc` and `pthread_create`.

Distributed Threads. Before starting applications, each machine in MAGI's cluster launches a loader. Each loader sets up identical virtual memory mappings according to the initial memory content received from the application's main thread. Then, each loader creates a node process which is essentially a clone of the main thread on each node. The node process acts as a server to spawn applications threads and handle DSM protocol requests from other nodes.

Memory Management. To avoid conflicts with the default memory allocator for auxiliary libraries (e.g., user-level RDMA driver), MAGI places the DSM heap in the hole of the virtual space of Linux processes. Applications allocate from the DSM heap by default, and thus all data are guaranteed to be consistent across nodes. Similar to NUMA systems, the initial ownership of the DSM heap is evenly partitioned across all nodes. Every node initially has the exclusive copies of pages for their own partition. The memory locality information is also exposed to applications. Threads start allocating memory from their own nodes and request from remote nodes when the local heap runs out. In our experiments, remote allocations are rare due to the partition schemes of applications.

The stack of each thread is also shared through DSM. However, similar to single-node multithreaded programs, accessing the stack data of one thread from

another is dangerous unless carefully managed (e.g., shared data structures on the stack of the main thread).

Unpinned Memory and Two-Sided RDMA. Since MAGI is built atop the InfiniBand network with RDMA capability, an optional design is leveraging one-sided RDMA operations for inter-node communication. However, we find it does not scale with the increasing DSM address space because this approach requires memory pinning to prevent the RDMA buffers from being swapped out or freed during RDMA accesses. Given that the DSM is partitioned across all nodes, it is infeasible to pin the whole DSM address space in a single node. LITE^[33] proposes to provide kernel support for RDMA operations and to register the whole physical memory as a single memory region (MR). The idea of supporting kernel-level one-sided operations in LITE can be applied to MAGI. However, it is still impossible to register the memory of the whole shared memory space because one machine only owns limited physical DRAM. Also, the kernel still has to handle virtual address mapping consistency of swapping cases specially for pinned memory.

Our design does not pin the memory buffers and let the kernel handle swapping of DSM pages normally. Unused pages will be swapped out to disk to make space for incoming pages. With memory unpinned, we use two-sided operations for inter-node data exchange.

NUMA-Aware Synchronization. Synchronization primitives provided by the pthread library are widely used in parallel applications. However, the cost of the DSM protocol renders those cache-based synchronization constructs inefficient. To enable efficient yet convenient synchronization for DSM applications, MAGI re-implements the pthread synchronization interfaces with a hierarchical design similar to lock cohorting^[1].

Take barrier as an example. Each barrier structure maintains a local barrier. The first thread that reaches the barrier of a node becomes the leader. The leader reports to the global lock server and blocks. Subsequent threads arriving at the barrier only block on the local barrier. When the barrier is globally released, the leader receives a message from the lock server and notifies the local participants. This approach reduces the message cost of sending release messages to all threads.

6 Experimental Evaluation

6.1 Setup

We use an 8-node cluster for evaluation. Each server in the cluster has two E5-2650 v4 processors (12 cores

and 30 MB L3 cache per CPU) operating constantly at 2.2 GHz (TurboBoost disabled), 128 GB RAM and a single-port Mellanox ConnectX-4 100 Gb/s card connected to the InfiniBand EDR network through Mellanox SB7890 InfiniBand Switch. Since MAGI dedicates two cores on each machine for the manager and per-node protocol handler, we limit the maximum number of cores available to applications on each machine to 20, making it a total of 160 cores.

6.2 Benchmarks

We use three sets of applications to evaluate the performance of MAGI.

PARSEC. Applications from PARSEC^[34] benchmark suite represent classic parallel workloads for SMP machines. Such applications feature fine-grained parallelism on a small input. Due to our current limitation on C++ support, we choose blackscholes and swaptions to examine how MAGI performs with small-scale parallel applications.

MapReduce. We re-implement a MapReduce framework for SMP based on the design of Phoenix^[26] instead of directly running Phoenix applications. The reason is that Phoenix fails to produce meaningful performance on MAGI due to its task queue design which intensively uses inefficient locks based on atomic instructions. We disable the task queue design and related work stealing mechanisms in our re-implementation. With the same interfaces, we use word count, matrix multiplication, and histogram for scalability evaluation.

Graph Algorithms. We use the classic PageRank algorithm based on Polymer^[6], a graph engine optimized for NUMA systems. We run PageRank for 20 iterations. Since MAGI is designed for NUMA-aware data-intensive application, Polymer is a good match. We will perform detailed analysis on the performance of the graph system on top of MAGI.

Polymer is a vertex-centric graph engine that provides two programming interfaces, namely EdgeMap and VertexMap. An application should provide two user-defined functions (UDFs) to be invoked by EdgeMap and VertexMap respectively. The major shared data structures are the graph topology and application-defined data. Graph topology tracks vertices and their neighbors by in-edges and out-edges. Application-defined data are processed by UDFs (e.g., the ranks of web pages for PageRank). In every iteration, Polymer traverses the set of active vertices, applies the user-defined EdgeMap function to each active

vertex’s neighbors, and updates the active vertex set in the graph topology. Polymer uses a NUMA-aware graph partitioning strategy. It partitions the vertices across all NUMA nodes such that the in-degrees or out-degrees of vertices are evenly distributed. Within each node, vertices are further partitioned across local cores.

Except evaluating the protocol-bypassing mechanism, applications undergo the same modifications mentioned in Section 5. For protocol-bypassing, we modify the iterations of Polymer such that every thread directly fetches the generated data of the previous iteration from all the other nodes at the beginning of the next iteration. Table 1 and Table 2 show the benchmark applications and the sizes of the inputs used in the benchmarks respectively.

Table 1. Benchmark Applications

Application	Input	Description
Blackscholes	10 M	PARSEC
Swaptions	$ns = 160, sm = 250\ 000$	PARSEC
Word count	4 GB	MapReduce
Matrix mult	2048×2048	MapReduce
Histogram	4 GB	MapReduce
PageRank	rMat24, twitter-2010, uk-2007-05	Graph algorithm

Table 2. Input Sizes of Graphs^[35–37]

Graph	Number of Vertices	Number of Edges
rMat24	16 777 216	268 000 000
twitter-2010	41 652 230	1 468 365 182
uk-2007-05	105 896 555	3 738 733 648

6.3 Optimization Breakdown

6.3.1 Effects of Speculative Faults

To evaluate the effects of speculative faults, we run PageRank on eight nodes with a total of 160 cores. We set the maximum speculative length to 8.

Fig.4(a) shows the average number of page faults for each node, while Fig.4(b) shows the average page fault time for each iteration. During our experiments, the statistics of page faults remains stable across 10 runs. For twitter-2010, speculation can ideally reduce the number of page faults by 86.27% compared with the baseline. But for uk-2007-5, speculation reduces the page faults by 81.97%. To explain the difference in page fault avoidance, we analyze the distribution of successful speculation lengths in Fig.5. The result for twitter-2010 has more successful 8-page speculations than the result for uk-2007-05. This implies increased page ownership interleaving within 8-page regions due to partitioning. We think this variance is acceptable, because

the partition strategy of single-node applications does not consider the page boundary in DSM systems and thus may cause false sharing.

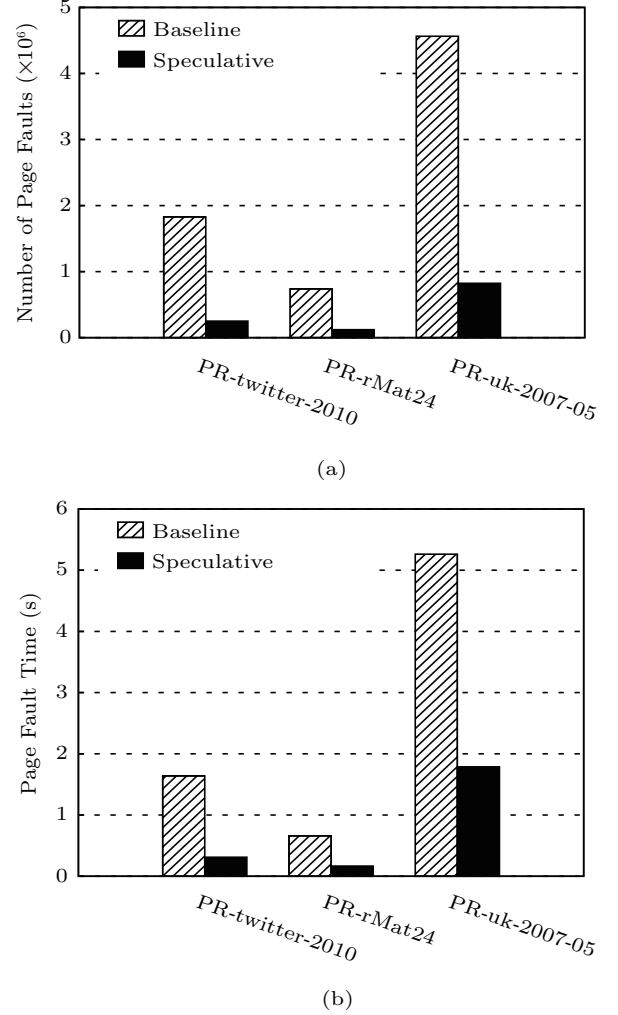


Fig.4. Reduction of page fault overhead. (a) Per-node page fault count. (b) Per-iteration page fault time.

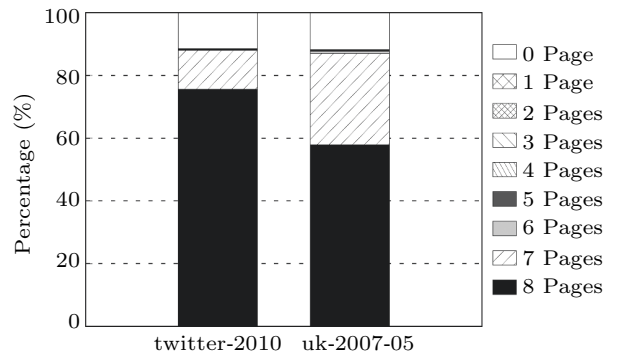


Fig.5. Distribution of the length of successful speculative faults.

6.3.2 Effects of Polling

To evaluate the performance gains of polling based protocol processing, we use a micro-benchmark to mea-

sure the time for managers to invalidate read-only copies. We launch two threads on two nodes with one thread repeatedly sending invalidation requests to the other in a closed loop. Fig.6 presents the distribution of round-trip latency of invalidating a page for one node. The duration comprises two RDMA messages and a TLB shutdown operation on the target node. The 50th percentile of latency for the interrupt-based event scheme is about 10 μ s higher than that of the polling scheme because of multiple context switches and in-directions of message queuing. Moreover, the event scheme has a very long tail for the 99th percentile of latency due to the non-determinism of scheduling. The polling scheme can effectively reduce the overall latency as well as the tail latency.

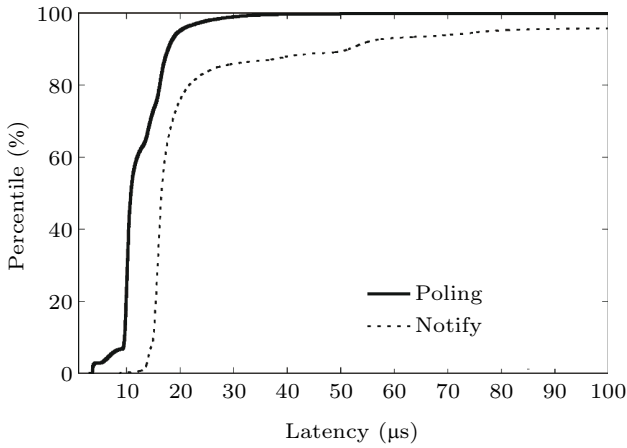


Fig.6. Round-trip latency of invalidating a page for one node.

6.3.3 Effects of Batched TLB Shutdown

We run PageRank on eight nodes and vary the number of threads per node. Fig.7 presents the cost of TLB shutdowns for changing page permissions. Since MAGI does not try to solve the scalability of IPIs, the TLB shutdown time still increases with the increasing number of threads per node. However, our batched approach can greatly reduce the per-page cost and thus ease the overall overhead of TLB shutdowns.

6.4 Scalability of Throughput

6.4.1 PARSEC and MapReduce

We measure the execution time for PARSEC benchmarks and MapReduce applications. The results are shown in Fig.8 and Fig.9. Blackscholes can trivially scale as it incurs little memory sharing. Swaptions has only hundreds of page faults per node; thus the overhead of the DSM protocol does not limit the scalability.

Due to the small memory footprint, the speculative approach has little effect. Word count has poor scalability on MAGI, because the reduce phase involves massive random accesses to remote nodes for the raw input data owned by the map phase in the modified state. Histogram and matrix multiplication have fewer page faults in the reduce phase; thus they are able to scale trivially.

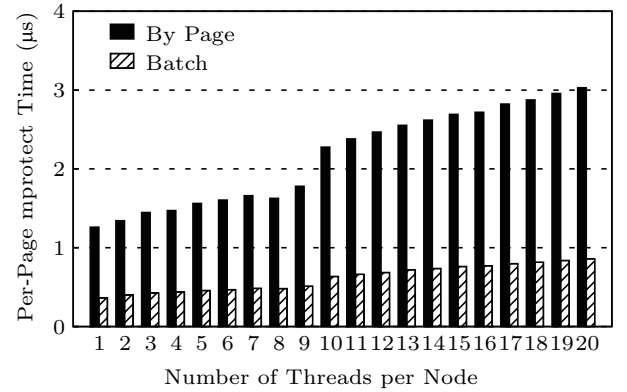
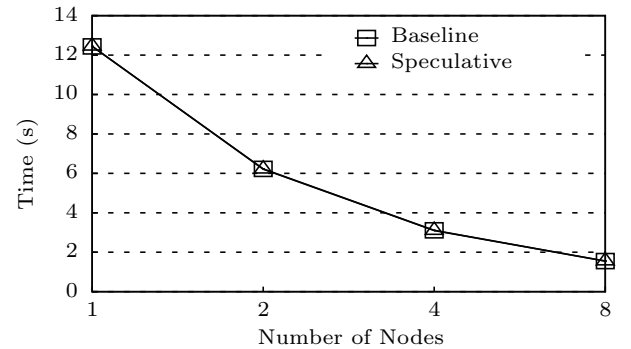
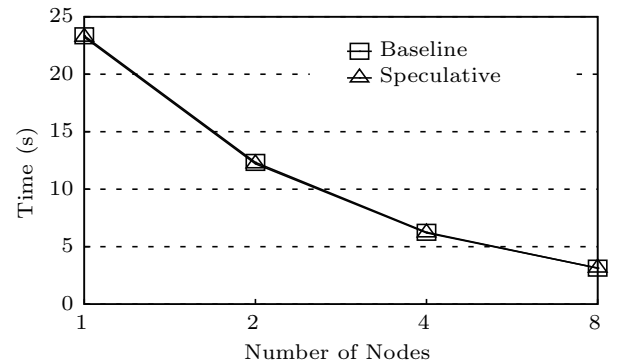


Fig.7. Per-page TLB shutdown time. mprotect is a function that performs “TLB shutdown”.



(a)



(b)

Fig.8. Scalability of PARSEC applications. (a) Blackscholes. (b) Swaptions.

From the results above, we conclude that the scalability of classic parallel applications with a small problem size depends on the contention of shared data.

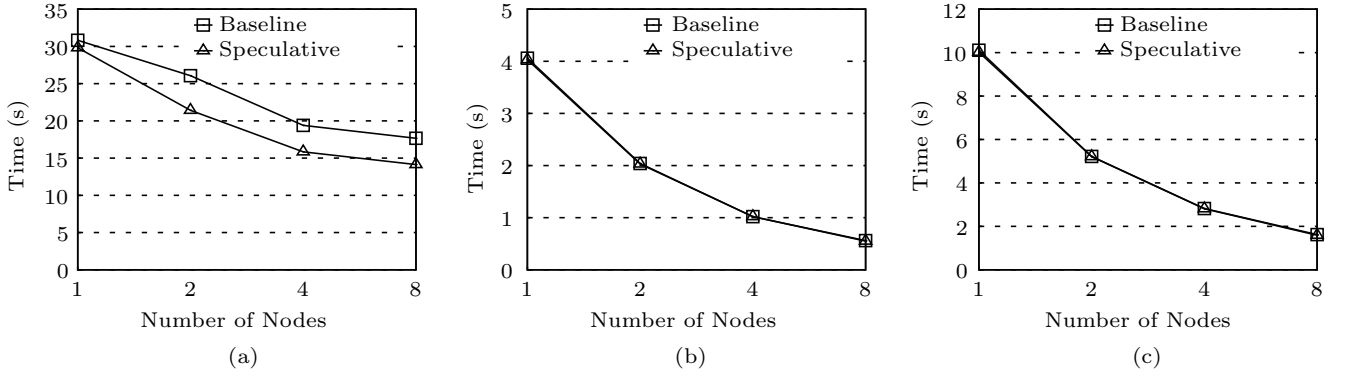


Fig.9. Scalability of MapReduce applications on MAGI. (a) Word count. (b) Histogram. (c) Matrix multiplication.

Since fine-grained data sharing is inherently hard to scale out with DSM, MAGI can scale out applications if they have low contention. Moreover, MAGI targets large-scale applications with coarse-grained sharing.

6.4.2 Graph Algorithms

Overall Performance. Table 3 shows the runtime of Polymer-DSM on eight nodes with different optimizations. Although the speculation can notably reduce the overhead of page faults, it is still not ideal because page faults are not fundamentally eliminated in the EdgeMap phase. The protocol-bypass optimization moves all the data synchronization to the beginning of each iteration and eliminates page faults in major phases of each iteration. However, the penalty of protocol-bypass is that all the shared data must be replicated due to the lack of knowledge of how many data are actually necessary for each worker. Also, the time for transferring data from remote nodes is unavoidable regardless of our optimizations.

Throughput Scalability. Instead of measuring the completion time of the whole algorithms, we take into account the throughput of EdgeMap whose execution time dominates the per-thread runtime. As mentioned in Subsection 6.2, the Polymer graph engine iteratively traverses vertices and applies user-defined functions to update application-defined data in the EdgeMap phase. Most of the page faults occur in this phase due to accessing the application data or topology data that are updated by other nodes in the previous iterations. However, the total runtime cannot properly reflect the scalability, because barriers may conceal the variance of completion time for different threads. On the contrary, the per-core throughput of applied updates can reveal the slowdown of computations due to the stalls introduced by page faults. Fig.10 shows the sorted per-core

throughput of the EdgeMap phase of PageRank. While we collect the data for a specific node, the results are similar across all nodes.

We observe significant variations in the per-core throughput on the same node. This can be partly attributed to the dual-socket structure of our test servers. Intra-node accesses in DSM may reference memory locations on remote NUMA nodes with increased latency. However, the main reason for the variations is the difference in the numbers of page faults triggered by threads. Polymer partitions a graph by evenly distributing the edges across nodes so that each node performs the same number of updates. But threads on one node differ in the number of remote page accesses. Some threads handle more inter-node edges while others handle more intra-node edges. Therefore, the DSM protocol with page-size unit may contradict balanced graph partitioning schemes.

Fig.10(a) and Fig.10(b) present the sorted per-core throughput for rMat24 and twitter-2010. MAGI notably improves the throughput of stragglers by reducing page faults and accelerating protocol handling. Specifically, the speculation mechanism can hide the difference in the number of triggered page faults, closing the throughput gap among threads. The protocol-bypassing technique further stabilizes throughput because all the data are fetched to local memory using direct-reads ahead of time and the EdgeMap phase is no longer interrupted by page faults. This is the theoretical throughput that the algorithms can achieve without DSM. Some threads achieve similar throughput in the baseline implementation as well as the protocol-bypassing versions. The reason is that these threads mainly handle intra-node edges, and do not incur the overhead of page faults.

Fig.10(c) presents the results of PageRank on the web graph uk-2007-05. This graph has far more vertices

Table 3. Runtime of Graph Algorithms with Polymer-DSM on Three Input Graphs

Algorithm	Graph	None	Speculative		Protocol-Bypass	
		Runtime (s)	Runtime (s)	Speedup (x)	Runtime (s)	Speedup (x)
PageRank	rMat24	29.6	11.7	2.53	5.65	5.24
	twitter-2010	60.8	26.8	2.27	20.00	3.04
	uk-2007-05	133.8	53.3	2.51	29.10	4.60
CC	rMat24	5.3	2.9	1.83	2.50	2.12
	twitter-2010	32.3	17.0	1.90	9.60	3.36
	uk-2007-05	59.8	30.7	1.95	15.40	3.88
SSSP	rMat24	48.8	31.3	1.56	6.60	7.39
	twitter-2010	48.7	29.1	1.67	8.40	5.80
	uk-2007-05	888.3	525.5	1.69	174.50	5.09
BFS	rMat24	2.0	1.6	1.25	0.40	5.00
	twitter-2010	17.0	13.2	1.29	3.50	4.86
	uk-2007-05	25.9	18.4	1.41	2.80	9.25
SpMV	rMat24	25.6	10.0	2.56	5.40	4.74
	twitter-2010	56.8	31.5	1.80	24.20	2.35
	uk-2007-05	116.0	46.7	2.48	30.10	3.85

Note: Algorithms used in the evaluation of runtime are PageRank, connected components (CC), single-source shortest paths (SSSP), breadth-first search (BFS), and sparse matrix-vector multiplication (SpMV).

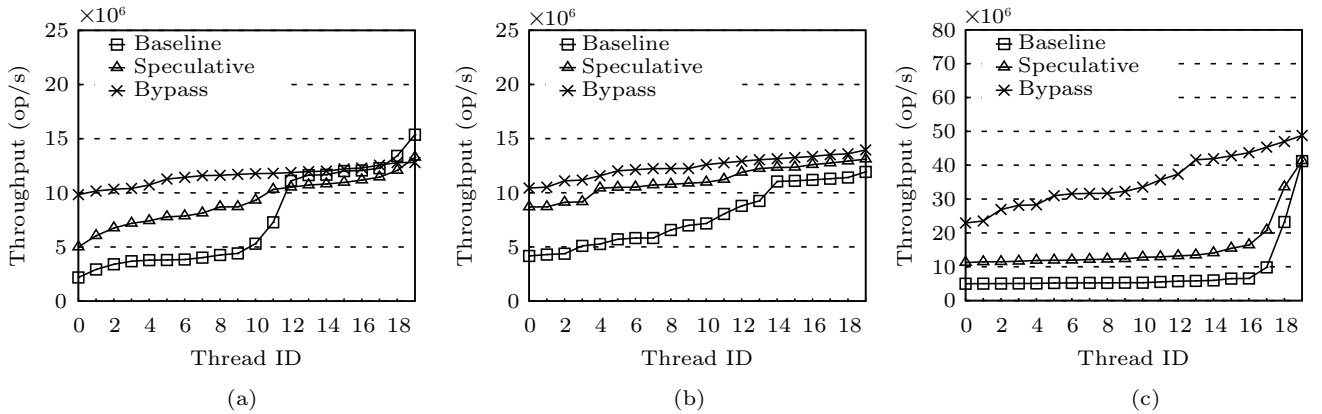


Fig.10. Sorted per-core throughput of the EdgeMap phase for PageRank. (a) rMat24. (b) twitter-2010. (c) uk-2007-05. Op: the number of operations.

and edges than the social graph twitter-2010. Even with speculation approach, there are still a large number of page faults. With protocol-bypassing operations eliminating page faults, the average throughput is improved by 2x to 3x.

Fig.11 further shows the throughput scalability with a varying number of nodes. For all the graphs, the baseline throughput drops with the increase of the number of nodes. Our optimizations can notably improve the scalability with sustained throughput for small and medium-sized graphs. However, for PageRank with uk-2007-05, the single-node deployment for baseline out-

performs all other cases. The reason is the way that we implement protocol-bypassing results in poor cache performance. Since we modify the iteration function to insert a data-fetch phase that directly copies the data of the previous iteration to a temporary DSM buffer, the EdgeMap function actually reads data from the temporary buffer and applies changes to the DSM. This implementation enlarges the memory footprint and increases LLC miss rate. Nevertheless, the optimized implementations finally outperform the single-node setting and achieve scalable performance with increased node count.

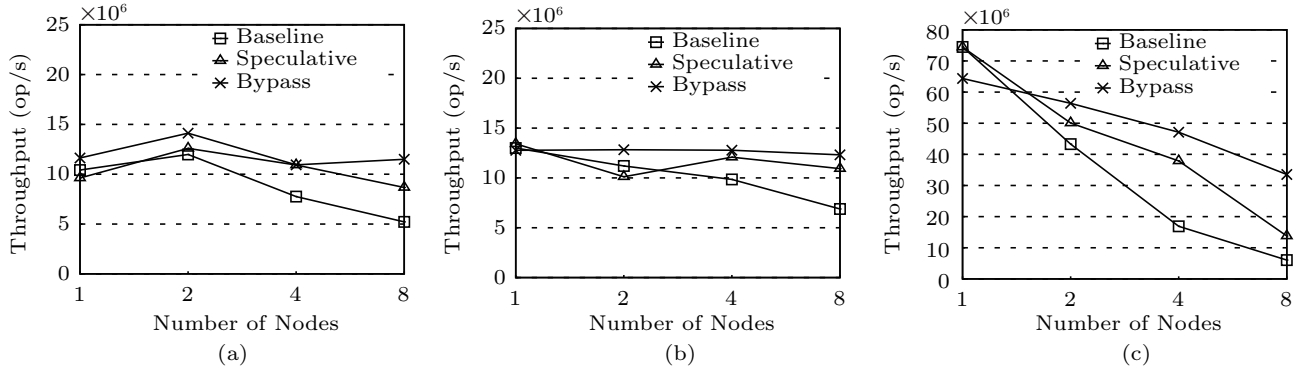


Fig.11. Scalability of per-core throughput of the EdgeMap phase for PageRank. (a) rMat24. (b) twitter-2010. (c) uk-2007-05. Op: the number of operations.

6.5 Overall Breakdown

Fig.12 shows the breakdowns of iteration execution time. The overhead of protocol-bypassing is represented as the overhead of page faults in the third bar. Due to the variation of execution time across threads, we calculate the weighted mean for each component by considering the total iteration time for each thread. Fig.12(a) confirms that speculation can effectively reduce the time for page faults, and Fig.12(b) shows that protocol-bypassing achieves better performance improvement than speculation in the case of large graphs. With protocol-bypassing operations, the data transfer time is much shorter. Note that the barrier time to synchronize iterations is also reduced because speculation can even up the difference of page fault counts. However, the imbalance of work on each thread still results in long barrier time. Fundamentally solving the imbalance problem requires the cooperation of applications and the DSM framework.

7 Limitations and Discussions

7.1 Page Swapping

For large-scale applications, node-local DRAM can be insufficient for the per-node working sets and unused data must be swapped out to make space for new data. The current implementation of MAGI only supports kernel-managed local swapping of DSM memory instead of inter-node swapping that keeps pages in the DSM instead of disks. An inter-node swapping approach requires redesigning the protocol to push pages to the home nodes and carefully determine the victim pages to be replaced. This is left for future work.

7.2 Fault Tolerance

MAGI aims at providing a virtual NUMA abstraction atop a well-connected small and medium-scale cluster, which runs NUMA-aware workloads at high speed. Such parallel applications can usually finish within a

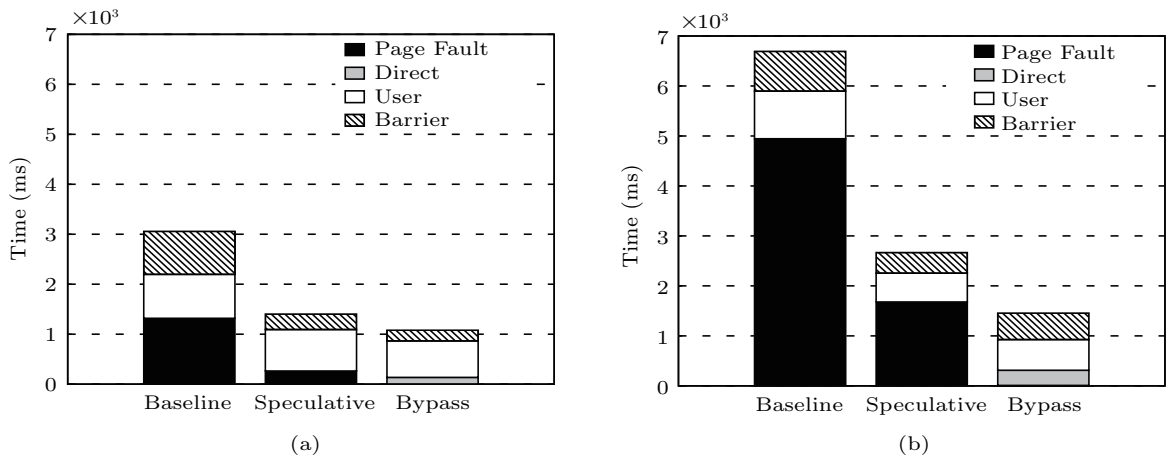


Fig.12. Breakdown of per-iteration time. (a) twitter-2010. (b) uk-2007-05.

few days or even minutes. In contrast, the mean time between failure (MTBF) for a 64-node cluster is 5.7 days assuming an MTBF of a machine to be one year. Hence, it should be fair enough to rerun an application upon failures. Another option is adding checkpoint/restart feature to support failure recovery, like those done in HPC area, which, however, would cause high runtime cost and is only cost-effective for large-scale long-running applications. Therefore, MAGI currently does not provide fault tolerance feature but instead simply reruns an application upon failure, as done in prior work^[20]. Implementing efficient fault tolerance and comparing the performance slowdown are left for future work.

7.3 Emerging Memory Technologies

Non-volatile memory technology (NVM) has been increasingly popular in recent years and many hybrid approaches are proposed to combine NVM with DRAM to provide high availability and performance. MAGI provides an efficient abstract layer of shared memory space, allowing cluster-scale memory disaggregation. Compared with other NVM-specific distributed shared memory systems, MAGI allows single-node applications to seamlessly scale out to a cluster.

7.4 Extending RDMA Operations

Currently, industry vendors like Mellanox are still continually evolving the RDMA features, including adding more queue pairs, providing dynamically connected transport^⑥. This can further enhance the scalability of RDMA operations and can help further scale MAGI. Meanwhile, there have been some proposals to provide more advanced one-sided RDMA operations, like multiple address RDMA atomic operations^[38] and one-sided RDMA atomic reads^[39]. Such proposals, if combined into commodity products, could help bring some benefits investigated in early hardware DSM proposals (e.g., [40, 41]) into MAGI by simplifying the coherence operations.

7.5 Reflection on Hardware Support for DSM

During the implementation of MAGI, we believe two important but missing features may accelerate DSM systems. One is invalidating remote TLB over RDMA

interconnect. This may be achieved by extending the agent for PCI-E to send special signals to the ring of on-chip interconnect and notify CPU cores to flush the TLB entries. If this feature is implemented, the overhead of fault handling can be further reduced because the fault handling process can totally eliminate the involvement of remote nodes. The other one is conditional execution of a batch of RDMA operations. This feature may help further reduce the number of PCI-E writes. Given the ability to decide whether the subsequent RDMA operation should be executed depending on the execution of previous requests, the number of round trips and MMIO writes can be reduced for better batching optimizations.

8 Related Work

There has been a long line of research in DSM. Compared with prior work, MAGI leverages RDMA primitives to implement a TSO-compatible virtual cc-NUMA system to scale out NUMA-aware applications. Here, we only discuss MAGI with most related work.

RDMA-Based Restricted DSM. The commodity availability of RDMA-capable NICs has stimulated a number of studies of RDMA to provide a restricted form of DSM (e.g., [18, 19, 42]). Specifically, FaRM^[18] provides a partitioned global address space (PGAS) using RDMA messaging and implements distributed in-memory transactions atop^[43]. DrTM^[19] further leverages the strong atomicity of hardware transactional memory with the strong consistency of one-sided RDMA operations with respect to local cache to implement distributed transactional memory for efficient database transactions. FaSST^[44] instead uses two-sided RDMA messages to implement low-overhead distributed transactions. In contrast, MAGI leverages RDMA operations to provide a full software DSM atop a cluster to reuse existing NUMA-based optimizations.

Software-Based DSM. Most software DSM systems can be rooted back to Ivy^[8], which provides sequential consistency. Later DSM systems explore various relaxed consistency models^[9,11,14] to improve the scalability. vNUMA^[10] further shows how to implement a transparent guest-physical address space to run virtual machines. However, it does not support SMP nodes. Other hypervisor-based implementations include vSMP^⑦, vFe^[45], and virtual multiprocessor^[46].

⑥ Mellanox[®] Technologies. Connect-IB[®] InfiniBand adapter cards. http://www.mellanox.com/related-docs/prod_adapter_cards/PB_Connect-IB.pdf, Oct. 2018.

⑦ ScaleMP[®]. Versatile SMP (vSMP) architecture. <https://www.scalemp.com/technology/versatile-smp-vsmp-architecture/>, Oct. 2018.

SoftFLASH^[15] and Shasta^[30] explore the design space of software DSM systems on SMP clusters and uncover the TLB shutdown scalability issue. However, they only are implemented on small SMP nodes and target small-scale parallel workloads.

Compared with prior work, MAGI leverages RDMA operations to implement sequential consistency across SMP servers, which notably reduces the overhead for data-intensive applications.

Exploitation of Fast Network. The poor performance of software-based memory consistency protocols has motivated the specialized hardware support for DSM systems. Cashmere^[41] proposes several hardware extensions to improve the performance of software DSM systems. Cashmere-2L^[16] exploits the remote-write feature of DEC Memory Channel^[47]. The SHRIMP^[40,48,49] project designed coherence protocols called Automatic Update Release Consistency (AURC) to implement atomic remote updates. AURC uses distributed information such as timestamps and write notices to maintain sharing information and relies on remote memory write to propagate shared data updates to home nodes. Yet, both SHRIMP and Cashmere implement release consistency while MAGI implements the sequential consistency which is compatible with current COTS server hardware.

The overlapped home-based LRC design^[49] uses a communication co-processor for twin generation, applying diffs and servicing remote requests, which are, however, not available in commodity clusters nowadays. MIT's Alewife project^[50] exploited multi-grain locality by considering multi-processor per node but showed some negative results of DSM: tightly-coupled multi-processor holds significant performance advantage over DSM. MAGI thus targets NUMA-aware applications pervasively available nowadays thanks to the multicore evolution.

Scale-out NUMA (soNUMA)^[51] provides an architectural design by integrating a remote memory controller into local coherency hierarchy to provide a form of PGAS for fast in-memory processing. SABRes^[39] provides an architectural mechanism called atomic object reads to atomically read a set of remote memory locations. Yet, there is currently no available implementation of both soNUMA and SABRes.

In contrast, MAGI revisits DSM on modern hardware and exploits RDMA operations to implement a DSM system with sequential consistency to scale out existing NUMA-optimized applications.

9 Conclusions

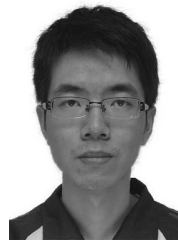
The commodity availability of cc-NUMA based multicore machine has forced a number of NUMA-aware optimizations for scaling up software. Yet, programmers have to use a different thinking on optimizations when scaling out NUMA-aware multicore software. This paper describes MAGI, which retrofits DSM to enable programmers to enjoy existing NUMA-aware optimizations to scale out software. MAGI overcomes the deficiency in classic software DSM by exploiting the fast network interfaces and memory access patterns in large-scale data-intensive applications. With several optimization techniques, MAGI can notably reduce the overhead and improve the scalability of page fault handling of DSM, a major source of overhead in DSM. Experimental evaluation using a set of NUMA-aware applications showed that MAGI notably reduces the cost for memory coherence. Detailed study showed that our optimizations notably reduce the coherence overhead. We will explore the integration of RDMA primitives and protocol operations to further reduce the overhead in the future.

References

- [1] Dice D, Marathe V J, Shavit N. Lock cohorting: A general technique for designing NUMA locks. In *Proc. the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, February 2012, pp.247-256.
- [2] Calciu I, Dice D, Lev Y, Luchangco V, Marathe V J, Shavit N. Numa-aware reader-writer locks. In *Proc. the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, February 2013, pp.157-166.
- [3] Boyd-Wickizer S, Kaashoek M F, Morris R, Zeldovich N. OpLog: A library for scaling update-heavy data structures. Technical Report, Massachusetts Institute of Technology, 2014. <https://dspace.mit.edu/handle/1721.1/89653>, September 2018.
- [4] Majo Z, Gross T R. (Mis)understanding the NUMA memory system performance of multithreaded workloads. In *Proc. the 2013 IEEE International Symposium on Workload Characterization*, September 2013, pp.11-22.
- [5] Majo Z, Gross T. R. A library for portable and composable data locality optimizations for NUMA systems. In *Proc. the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, February 2015, pp.227-238.
- [6] Zhang K, Chen R, Chen H. NUMA-aware graph-structured analytics. In *Proc. the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, February 2015, pp.183-193.
- [7] Calciu I, Sen S, Balakrishnan M, Aguilera M K. Black-box concurrent data structures for NUMA architectures. In *Proc. the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*, April 2017, pp.207-221.

- [8] Li K, Hudak P. Memory coherence in shared virtual memory systems. *ACM Transactions on Computer Systems*, 1989, 7(4): 321-359.
- [9] Bennett J K, Carter J B, Zwaenepoel W. Munin: Distributed shared memory based on type-specific memory coherence. In *Proc. the 2nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, March 1990, pp.168-176.
- [10] Chapman M, Heiser G. vNUMA: A virtual shared-memory multiprocessor. In *Proc. the 2009 USENIX Annual Technical Conference*, June 2009, pp.349-362.
- [11] Keleher P, Cox A L, Dwarkadas S, Zwaenepoel W. TreadMarks: Distributed shared memory on standard workstations and operating systems. In *Proc. the USENIX Winter 1994 Technical Conference*, January 1994, pp.115-132.
- [12] Fleisch B, Popek G. Mirage: A coherent distributed shared memory design. In *Proc. the 12th ACM Symposium on Operating Systems Principles*, December 1989, pp.211-223.
- [13] Iftode L, Singh J P, Li K. Scope consistency: A bridge between release consistency and entry consistency. In *Proc. the 8th Annual ACM Symposium on Parallel Algorithms and Architectures*, June 1996, pp.277-287.
- [14] Bershad B N, Zekauskas M J, Sawdon W A. The Midway distributed shared memory system. In *Proc. Digest of Papers. Comcon Spring*, February 1993, pp.528-537.
- [15] Erlichson A, Nuckolls N, Chesson G, Hennessy J. SoftFLASH: Analyzing the performance of clustered distributed virtual shared memory. In *Proc. the 7th International Conference on Architectural Support for Programming Languages and Operating Systems*, October 1996, pp.210-220.
- [16] Stets R, Dwarkadas S, Hardavellas N, Hunt G, Konthanas L, Parthasarathy S, Scott M. Cashmere-2L: Software coherent shared memory on a clustered remote-write network. In *Proc. the 16th ACM Symposium on Operating Systems Principles*, October 1997, pp.170-183.
- [17] Charles P, Grothof C, Saraswat V, Donawa C, Kielstra A, Ebcioglu K, von Praun C, Sarkar V. X10: An object-oriented approach to non-uniform cluster computing. In *Proc. the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, October 2005, pp.519-538.
- [18] Dragojević A, Narayanan D, Hodson O, Castro M. FaRM: Fast remote memory. In *Proc. the 11th USENIX Symposium on Networked Systems Design and Implementation*, April 2014, pp.401-414.
- [19] Wei X, Shi J, Chen Y, Chen R, Chen H. Fast in-memory transaction processing using RDMA and HTM. In *Proc. the 25th Symposium on Operating Systems Principles*, October 2015, pp.87-104.
- [20] Nelson J, Holt B, Myers B, Briggs P, Ceze L, Kahan S, Oskin M. Latency-tolerant software distributed shared memory. In *Proc. the 2015 USENIX Annual Technical Conference*, July 2015, pp.291-305.
- [21] Kalia A, Kaminsky M, Andersen D G. Design guidelines for high performance RDMA systems. In *Proc. the 2016 USENIX Annual Technical Conference*, June 2016, pp.437-450.
- [22] Kumar M, Maass S, Kashyap S, Vesely J, Yan Z, Kim T, Bhattacharjee A, Krishna T. LATR: Lazy translation coherence. In *Proc. the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems*, March 2018, pp.651-664.
- [23] Dennard R H, Gaensslen F H, Rideout V L, Bassous E, LeBlanc A R. Design of ion-implanted MOSFET's with very small physical dimensions. *IEEE Journal of Solid-State Circuits*, 1974, 9(5): 256-268.
- [24] Esmaeilzadeh H, Blem E, Amant R S, Sankaralingam K, Burger D. Dark silicon and the end of multicore scaling. In *Proc. the 38th International Symposium on Computer Architecture*, June 2011, pp.365-376.
- [25] Borkar S. Thousand core chips: A technology perspective. In *Proc. the 44th Annual Design Automation Conference*, June 2007, pp.746-749.
- [26] Ranger C, Raghuraman R, Penmetsa A, Bradski G, Kozyrakis C. Evaluating MapReduce for multi-core and multiprocessor systems. In *Proc. the 13th International Symposium on High-Performance Computer Architecture*, February 2007, pp.13-24.
- [27] Chen R, Chen H, Zang B. Tiled-MapReduce: Optimizing resource usages of data-parallel applications on multicore with tiling. In *Proc. the 19th International Conference on Parallel Architectures and Compilation Techniques*, September 2010, pp.523-534.
- [28] Zhu X, Chen W, Zheng W, Ma X. Gemini: A computation-centric distributed graph processing system. In *Proc. the 12th USENIX Symposium on Operating Systems Design and Implementation*, November 2016, pp.301-316.
- [29] Guo C, Wu H, Deng Z, Soni G, Ye J, Padhye J, Lipshteyn M. RDMA over commodity Ethernet at scale. In *Proc. the 2016 ACM SIGCOMM Conference*, August 2016, pp.202-215.
- [30] Scales D J, Gharachorloo K, Thekkath C A. Shasta: A low overhead, software-only approach for supporting fine-grain shared memory. In *Proc. the 7th International Conference on Architectural Support for Programming Languages and Operating Systems*, October 1996, pp.174-185.
- [31] Karlsson M, Stenström P. Performance evaluation of a cluster-based multiprocessor built from ATM switches and bus-based multiprocessor servers. In *Proc. the 2nd International Symposium on High-Performance Computer Architecture*, February 1996, pp.4-13.
- [32] Gaud F, Lepers B, Decouchant J, Funston J, Fedorova A, Quéma V. Large pages may be harmful on NUMA systems. In *Proc. the 2014 USENIX Annual Technical Conference*, June 2014, pp.231-242.
- [33] Tsai S, Zhang Y. LITE kernel RDMA support for datacenter applications. In *Proc. the 26th Symposium on Operating Systems Principles*, October 2017, pp.306-324.
- [34] Bienia C, Kumar S, Singh J P, Li K. The PARSEC benchmark suite: Characterization and architectural implications. In *Proc. the 17th International Conference on Parallel Architectures and Compilation Techniques*, October 2008, pp.72-81.
- [35] Shun J, Blelloch G E. Ligra: A lightweight graph processing framework for shared memory. In *Proc. the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, February 2013, pp.135-146.

- [36] Boldi P, Vigna S. The WebGraph framework I: Compression techniques. In *Proc. the 13th International Conference on World Wide Web*, May 2004, pp.595-602.
- [37] Boldi P, Rosa M, Santini M, Vigna S. Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks. In *Proc. the 20th International Conference on World Wide Web*, March 2011, pp.587-596.
- [38] Raikin S, Liss L, Shachar A, Bloch N, Kagan M. Remote transactional memory. US Patent US20150269116, 2015. <http://www.freepatentsonline.com/20150269116.pdf>, September 2018.
- [39] Daglis A, Ustiugov D, Novaković S, Bugnion E, Falsafi B, Grot B. SABRes: Atomic object reads for in-memory rack-scale computing. In *Proc. the 49th Annual ACM/IEEE International Symposium on Microarchitecture*, October 2016, Article No. 6.
- [40] Blumrich M A, Li K, Alpert R, Dubnicki C, Felten E W, Sandberg J. Virtual memory mapped network interface for the SHRIMP multicomputer. In *Proc. the 21st Annual International Symposium on Computer Architecture*, April 1994, pp.142-153.
- [41] Kontothanassis L I, Scott M L. Using memory-mapped network interfaces to improve the performance of distributed shared memory. In *Proc. the 2nd International Symposium on High-Performance Computer Architecture*, February 1996, pp.166-177.
- [42] Kalia A, Kaminsky M, Andersen D G. Using RDMA efficiently for key-value services. In *Proc. the 2014 ACM Conference on SIGCOMM*, August 2014, pp.295-306.
- [43] Dragojević A, Narayanan D, Nightingale E B, Renzelmann M, Shamis A, Badam A, Castro M. No compromises: Distributed transactions with consistency, availability, and performance. In *Proc. the 25th ACM Symposium on Operating Systems Principles*, October 2015, pp.54-70.
- [44] Kalia A, Kaminsky M, Andersen D G. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. the 12th USENIX Symposium on Operating Systems Design and Implementation*, November 2016, pp.185-201.
- [45] Vasilevsky A, Lively D, Ofsthun S. Linux® virtualization on Virtual Iron™ VFe. In *Proc. the 2005 Ottawa Linux Symposium*, July 2005, pp.235-250.
- [46] Kaneda K, Oyama Y, Yonezawa A. A virtual machine monitor for providing a single system image. In *Proc. the 17th IPSJ Computer System Symposium*, November 2005, pp.3-12.
- [47] Gillett R B. Memory channel network for PCI. *IEEE Micro*, 1996, 16(1): 12-18.
- [48] Blumrich M A, Alpert R D, Chen Y et al. Design choices in the SHRIMP system: An empirical study. In *Proc. the 25th Annual International Symposium on Computer Architecture*, June 1998, pp.330-341.
- [49] Zhou Y, Iftode L, Li K. Performance evaluation of two home-based lazy release consistency protocols for shared virtual memory systems. In *Proc. the 2nd USENIX Symposium on Operating Systems Design and Implementation*, October 1996, pp.75-88.
- [50] Yeung D, Kubiawicz J, Agarwal A. MGS: A multigrain shared memory system. In *Proc. the 23rd Annual International Symposium on Computer Architecture*, May 1996, pp.44-55.
- [51] Novakovic S, Daglis A, Bugnion E, Falsafi B, Grot B. Scaleout NUMA. In *Proc. the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, March 2014, pp.3-18.



Yang Hong is currently a Ph.D. candidate at Shanghai Key Laboratory for Scalable Computing Systems, Shanghai Jiao Tong University, Shanghai. He received his B.S. degree in software engineering from Shanghai Jiao Tong University, Shanghai, in 2013. His research interests include computer architecture, virtualization, parallel computing, and networked systems.



Yang Zheng is a Master student in Shanghai Jiao Tong University, Shanghai. He received his B.S. degree in software engineering from Shanghai Jiao Tong University, Shanghai, in 2016. His research interests are parallel computing, computer network and virtualization.



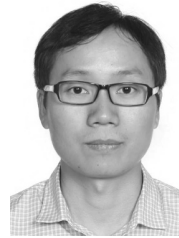
Fan Yang is a Master student in Shanghai Jiao Tong University, Shanghai. He received his B.S. degree in software engineering from Shanghai Jiao Tong University, Shanghai, in 2018. His research interests are graph computing and computer network.



Bin-Yu Zang is a professor in Shanghai Key Laboratory for Scalable Computing Systems, Shanghai Jiao Tong University, Shanghai. He received his B.S. degree, M.S. degree and Ph.D. degree in computer science from Fudan University, Shanghai, in 1987, 1990 and 1999 respectively. His research interests are compilers, computer architecture, virtualization, system security, and data-parallel computing.



Hai-Bing Guan is a professor of Shanghai Key Laboratory for Scalable Computing Systems, Shanghai Jiao Tong University, Shanghai. He received his Ph.D. degree in computer engineering from Tongji University, Shanghai, in 1999, and joined Shanghai Jiao Tong University, Shanghai, in 2002. He is in charge of the Shanghai Key Laboratory of Scalable Computing and Systems, Shanghai. His research interests include computer systems, network security, network storage, green IT and cloud computing.



Hai-Bo Chen is a professor in Shanghai Key Laboratory for Scalable Computing Systems, Shanghai Jiao Tong University, Shanghai. He received his Ph.D. degree in computer science from Fudan University, Shanghai, in 2009. Before joining Shanghai Jiao Tong University, he worked as an assistant processor at Parallel Processing Institute, Fudan University, Shanghai. His research interests include computer security, operating systems, and distributed systems.